SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5117

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Assessment Tool Weightage**:** 10%

QUESTIONS: **Total Marks: 50**

Annexure A

Debugging & Optimization Task

Code of Conduct:

I Namburi Sai Yashaswini(192425250(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: yashaswini

Annexure A
Debugging & Optimisation Task

1. Debugging & Optimization of Symmetric Encryption (DES / 3DES Implementation)
A Python program implementing **DES and 3DES encryption** produces inconsistent ciphertext for identical inputs across multiple runs.

A. Identify and fix logical and implementation errors in key scheduling and permutation steps.
B. Optimize the code to reduce execution time for large input datasets (≥10 MB).
C. Justify whether replacing DES/3DES with a modern cipher improves both security and performance.

2. Performance Analysis of Block Cipher Modes of Operation A Python script implements **AES-like block cipher modes (ECB, CBC, CFB, OFB)** but suffers from high latency and memory inefficiency.

A. Debug incorrect padding and IV handling causing decryption failures.
B. Refactor the code to minimize redundant computations and memory usage.
C. Compare and optimize the implementation to determine the most efficient mode for real-time secure communication.

3. Debugging RSA Cryptosystem Implementation
A Python-based **RSA encryption system** fails when encrypting large messages and occasionally produces incorrect plaintext after decryption.

A. Identify issues in prime generation, key generation, and modular exponentiation.
B. Optimize the algorithm using efficient techniques (e.g., fast exponentiation, Chinese Remainder Theorem).
C. Evaluate the trade-offs between key size, security, and computational performance.

4. Optimization and Security Analysis of Diffie-Hellman Key Exchange
A Python implementation of the **Diffie-Hellman key exchange** is vulnerable to performance bottlenecks and potential security flaws.

A. Debug incorrect shared key generation due to improper modular arithmetic.
B. Optimize exponentiation operations for large prime values.
C. Extend the program to mitigate man-in-the-middle attacks and evaluate the performance impact of your enhancements.

5. Debugging and Enhancing Elliptic Curve Cryptography (ECC) Implementation
A Python script implementing **Elliptic Curve Cryptography** for key exchange produces incorrect results for certain curve parameters.

A. Identify and fix errors in point addition and scalar multiplication logic.
B. Optimize the implementation using efficient algorithms (e.g., double-and-add).

C. Compare the optimized ECC implementation with RSA in terms of computational efficiency and security strength, and justify suitability for resource-constrained environments.

Scenario 3: Debugging RSA Cryptosystem Implementation

> **Scenario:** A Python-based RSA encryption system fails when encrypting large messages and occasionally produces incorrect plaintext after decryption. (1) Identify issues in prime generation, key generation and modular exponentiation. (2) Optimize the algorithm using efficient techniques (e.g. fast exponentiation, Chinese Remainder Theorem). (3) Evaluate the trade-offs between key size, security and computational performance.

1. Understanding the Problem

RSA encrypts an integer m as c = m^e mod n and recovers it as m = c^d mod n, where n = p·q, e is the public exponent and d is the inverse of e modulo the group order. For this to work, p and q must really be prime, e must be invertible, every message integer must be smaller than n, and the conversion between bytes and integers must be lossless. The reported symptoms show that at least one of these conditions is violated.

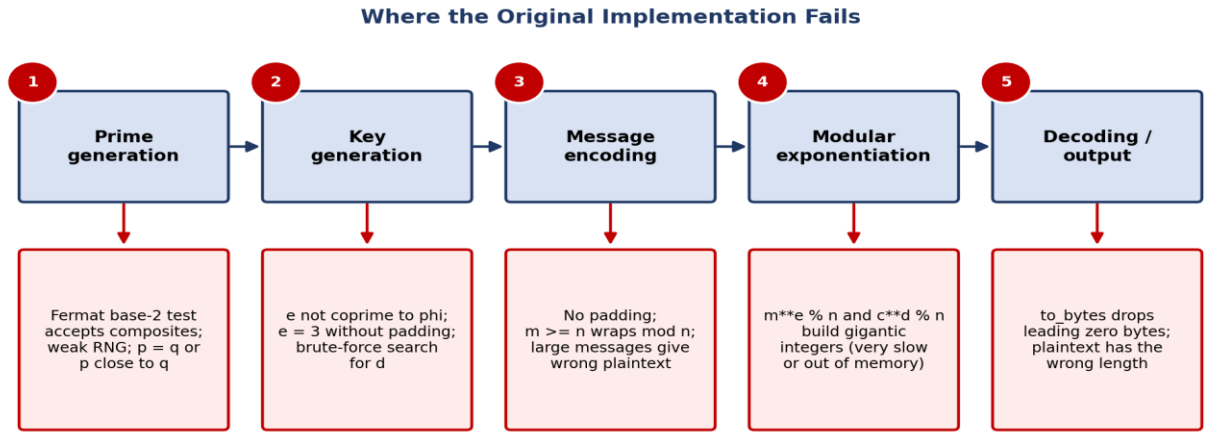


Figure 1: Stages of the RSA pipeline and the faults found at each stage

Table 1: Symptoms mapped to causes

Reported symptom	Likely root causes found in the code
Fails on large messages	No padding or block splitting, so m >= n and the plaintext wraps modulo n; m**e and c**d create enormous intermediate integers
Occasionally wrong plaintext	A composite number accepted as prime, e not coprime to the group order, floating-point arithmetic, and leading zero bytes dropped when converting back to bytes
Slow key generation and decryption	Brute-force search for d, naive (m ** d) % n, no CRT

Reported symptom	Likely root causes found in the code
Insecure even when it works	Textbook RSA without padding, tiny e, non-cryptographic RNG, primes that can be equal or close together

Part 1: Identifying the Issues

2. Original Faulty Implementation

The listing below reproduces the structure of the faulty program. Each defect is tagged with a BUG number that is explained in the following sections and demonstrated with an actual test run.

Listing 1: Original (faulty) implementation

```
import random, math
def is_prime(n):                    # BUG 1: Fermat test, base 2 only
    return n > 1 and pow(2, n - 1, n) == 1
def gen_prime(bits):
    while True:
        n = random.getrandbits(bits)    # BUG 2: Mersenne Twister, not a CSPRNG; BUG 3:
top/low bits not forced
        if is_prime(n): return n
def keygen(bits=512):
    p = gen_prime(bits // 2); q = gen_prime(bits // 2)   # BUG 4: p may equal q or be close
    n = p * q
    phi = (p - 1) * (q - 1)
    e = 3                           # BUG 5: gcd(e, phi) never checked; tiny e without padding
    d = 1
    while (e * d) % phi != 1: d += 1    # BUG 6: brute-force inverse, O(phi)
    return (e, n), (d, n)
def encrypt(msg, pub):
    e, n = pub
    m = int.from_bytes(msg, "big")     # BUG 7: no padding, m may be >= n
    return (m ** e) % n                # BUG 8: huge intermediate value
def decrypt(c, priv):
    d, n = priv
    m = (c ** d) % n                   # BUG 9: c**d is astronomically large
    return m.to_bytes((m.bit_length() + 7) // 8, "big")   # BUG 10: leading zero bytes lost
```

3. Issues in Prime Generation

Table 2: Prime generation defects

Bug	Problem	Consequence	Evidence from test run
1	Primality tested with the Fermat test to base 2 only	Composite numbers called pseudoprimes pass the test, so n is not a product of two primes and phi is wrong; decryption fails for many messages	22 composites below 10,000 pass (e.g. 341, 561, 645, 1105, 1387, 1729); Miller-Rabin accepts none
2	random.getrandbits() used as the randomness source	The Mersenne Twister is predictable after observing 624 outputs, so an attacker can reproduce p and q	Design flaw (use secrets / os.urandom)
3	Top bits and the low bit not forced	n can have fewer bits than intended, and even candidates waste time	Key size is not guaranteed
4	p and q may be equal or close together	If p = q, n = p^2 is easy to factor; if p and q are close, Fermat's factorisation finds them quickly	1024-bit n with prime gap 72 factored in 1 step (0.012 ms)

Demonstration of Bug 1. Using the composite 341 = 11 × 31 (which passes the base-2 Fermat test) together with the prime 179 gives n = 61039. With e = 3 and d computed from (p-1)(q-1), 1936 of 2998 test messages (64.6%) decrypt to the wrong value. This is the "occasionally incorrect plaintext" symptom: the key is valid-looking but mathematically broken.

4. Issues in Key Generation

Table 3: Key generation defects

Bug	Problem	Consequence	Evidence from test run
5	gcd(e, phi) is never checked; e is fixed at 3	If gcd(e, phi) > 1 there is no inverse d. Example: p = 61, q = 53 gives phi = 3120 and gcd(3, 3120) = 3	The while-loop for d never terminates
5b	Very small e (3) with no padding	If m^3 < n the ciphertext is the plain integer m^3, so an integer cube root recovers the message with no private key	Recovered "PIN:4821" from the ciphertext alone
6	d found by brute-force loop up to phi	Needs up to phi iterations; infeasible for real key sizes	phi of 25 bits: 688.7 ms vs 0.015 ms with extended Euclid; for 2048 bits it would never finish

5. Issues in Modular Exponentiation and Message Handling

Table 4: Modular exponentiation and message-encoding defects

Bug	Problem	Consequence	Evidence from test run
7	No padding; message integer m can be >= n	Values wrap modulo n, so the decrypted result is m mod n. Example: n = 3233, m = 4000 returns 767	A 200-byte message (1599 bits) under a 1024-bit key is not recovered
8	(m ** e) % n computes the full power first	The intermediate has about e·log2(m) bits; huge memory and time	At a 21-bit modulus naive decryption takes 429.6 ms vs 0.013 ms for square-and-multiply

Bug	Problem	Consequence	Evidence from test run
9	(c ** d) % n with a full-size d	The result has d·log2(c) bits, which is astronomically large (about 2^1034 bits for 1024-bit keys); the program stalls or runs out of memory	Not computable at real sizes
9b	Floating-point power used for small keys	float(65)**17 loses precision, so the remainder is wrong: exact 2790, float result 887	Silent wrong answers
10	to_bytes with length (bit_length+7)//8	Leading zero bytes vanish: 00004869 comes back as 4869	Plaintext length changes

> **Summary of diagnosis:** The failure on large messages is caused by Bugs 7 to 9 (no padding or block splitting, so m >= n, and an unusable exponentiation method). The occasional wrong plaintext is caused by Bugs 1 and 5 (a composite modulus factor or non-invertible e), Bug 9b (floating-point arithmetic) and Bug 10 (lost leading zero bytes).

Part 2: Optimising and Correcting the Algorithm

6. Reliable Prime Generation

- **Cryptographically secure randomness:** candidates come from Python's secrets module.
- **Forced bit pattern:** the top two bits and the lowest bit are set, so each prime has exactly the required length, the product n has exactly the required length, and no even candidates are tested.
- **Trial division by small primes, then Miller-Rabin:** a sieve of primes up to 2000 removes most composites cheaply. Miller-Rabin then writes n - 1 = 2^s·d and tests random bases a. A composite survives one round with probability at most 1/4, so 16 or more rounds make an error negligible. Unlike the Fermat test, it has no Carmichael-type blind spots.
- **Distinct and well-separated primes:** regenerate if p = q or |p - q| < 2^(bits/2 - 100).

Listing 2: Miller-Rabin and prime generation (corrected)

```
def is_probable_prime(n, rounds=None):
    if n < 2: return False
    for sp in SMALL_PRIMES:
        if n == sp: return True
        if n % sp == 0: return False
    bits = n.bit_length()
```

```
    rounds = rounds or (40 if bits <= 256 else 16 if bits <= 1024 else 8)
    d, s = n - 1, 0
    while d % 2 == 0: d //= 2; s += 1
    for _ in range(rounds):
        a = secrets.randbelow(n - 3) + 2
        x = mod_exp(a, d, n)
        if x in (1, n - 1): continue
        for _ in range(s - 1):
            x = x * x % n
            if x == n - 1: break
        else:
            return False
    return True

def gen_prime(bits):
    while True:
        c = secrets.randbits(bits) | (1 << (bits - 1)) | (1 << (bits - 2)) | 1
        if is_probable_prime(c): return c
```

7. Correct Key Generation

- **Fixed public exponent e = 65537 (2^16 + 1):** large enough to avoid the small-exponent attacks, yet its binary form has only two 1-bits, so encryption needs just 16 squarings and 1 multiplication.
- **Check gcd(e, lambda) = 1** and regenerate the primes if it fails. Using the Carmichael value lambda = lcm(p-1, q-1) gives the smallest valid private exponent.
- **Extended Euclidean algorithm** computes d = e^-1 mod lambda in O(log^2) time instead of a brute-force loop.
- **Reject small d** (d should be at least about n^0.5) to stay clear of Wiener-type attacks.
- **Precompute CRT parameters** dp, dq and qinv once, at key generation.

Listing 3: Extended Euclid and key generation (corrected)

```
def egcd(a, b):
    x0, x1, y0, y1 = 1, 0, 0, 1
    while b:
        q, a, b = a // b, b, a % b
        x0, x1 = x1, x0 - q * x1
```

```
        y0, y1 = y1, y0 - q * y1
    return a, x0, y0

def mod_inv(a, m):
    g, x, _ = egcd(a, m)
    if g != 1: raise ValueError("no modular inverse (gcd != 1)")
    return x % m

def generate_keypair(bits=2048, e=65537):
    while True:
        p, q = gen_prime(bits // 2), gen_prime(bits // 2)
        if p == q or abs(p - q) < (1 << (bits // 2 - 100)): continue   # distinct, not close
        n = p * q
        if n.bit_length() != bits: continue
        lam = (p - 1) * (q - 1) // math.gcd(p - 1, q - 1)              # Carmichael lambda
        if math.gcd(e, lam) != 1: continue
        d = mod_inv(e, lam)
        if d.bit_length() < bits // 2: continue
        return (e, n), {"n": n, "e": e, "d": d, "p": p, "q": q,
                        "dp": d % (p - 1), "dq": d % (q - 1), "qinv": mod_inv(q, p)}
```

8. Fast Modular Exponentiation (Square-and-Multiply)

Instead of computing m^e and then reducing, the exponent is read bit by bit and every intermediate result is reduced modulo n, so numbers never exceed 2·log2(n) bits. The number of multiplications falls from about e (exponential in the bit length of e) to at most 2·log2(e). Worked example for 7^13 mod 33 (13 = 1101 in binary, processed from the least significant bit):

Table 5: 7^13 mod 33 by square-and-multiply (result = 13)

Step	Exponent bit	Base (squared each step)	Result before	Result after
1	1	7	1	7
2	0	16 (7^2 mod 33)	7	7
3	1	25 (16^2 mod 33)	7	10 (7 × 25 mod 33)

Step	Exponent bit	Base (squared each step)	Result before	Result after
4	1	31 (25^2 mod 33)	10	13 (10 × 31 mod 33)

Listing 4: Square-and-multiply

```
def mod_exp(base, exp, mod):          # square-and-multiply, O(log exp)
    result, base = 1, base % mod
    while exp:
        if exp & 1: result = result * base % mod
        base = base * base % mod
        exp >>= 1
    return result
```

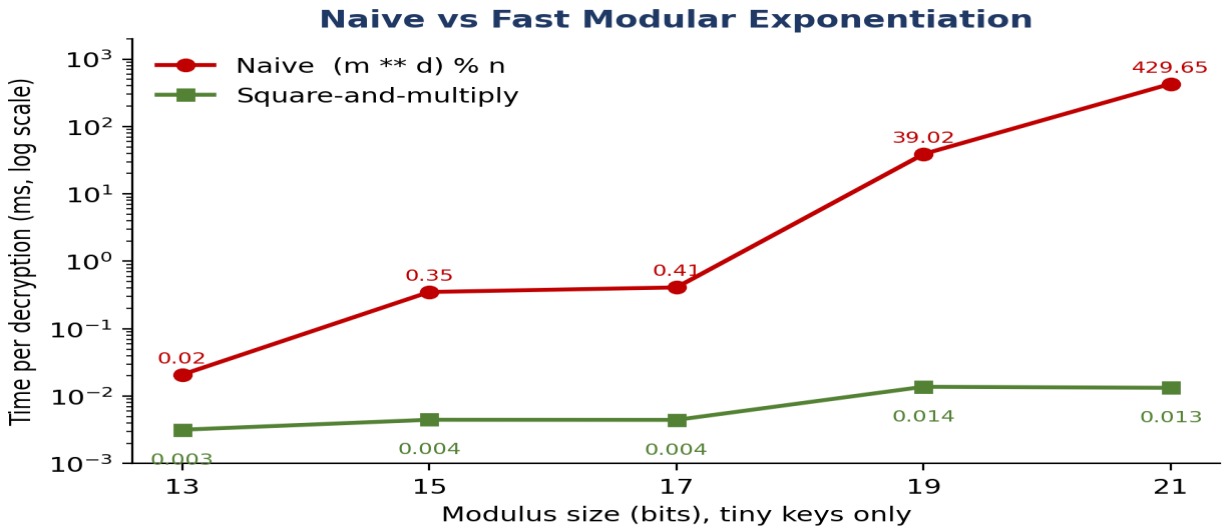


Figure 2: Naive versus square-and-multiply decryption time on tiny keys (measured)

The measurements show the effect even at toy sizes: at a 21-bit modulus the naive method already needs about 302,273 multiplications for d, while square-and-multiply needs about 38. For a 2048-bit key the naive method would need about 2^2048 multiplications and is impossible, whereas square-and-multiply needs roughly 3000.

9. Chinese Remainder Theorem (CRT) Decryption

The owner of the private key knows p and q, so decryption can be performed modulo p and modulo q separately and then recombined. Both exponentiations use half-size numbers, so each costs about one eighth of a full-size exponentiation, which gives a theoretical speed-up of about four times.

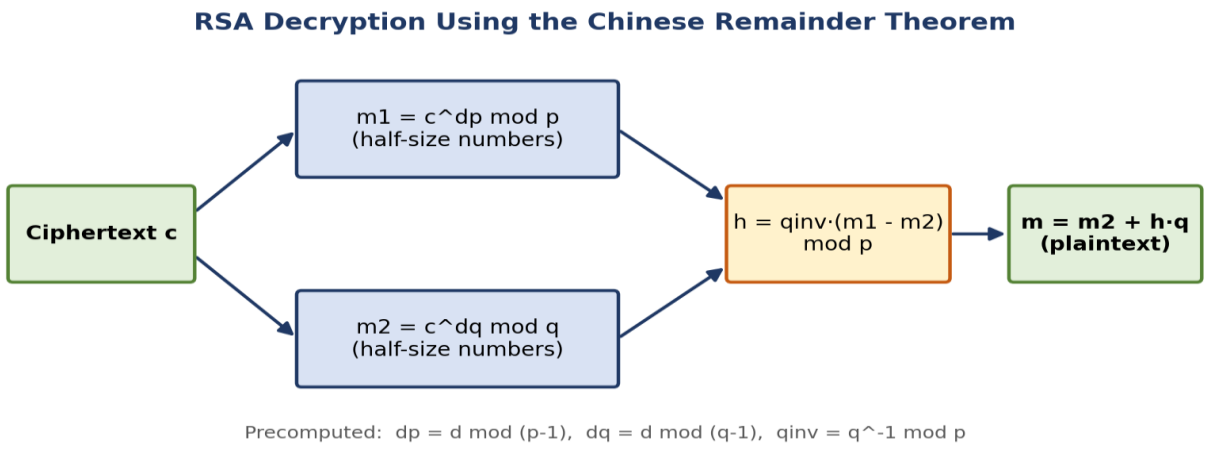


Figure 3: CRT-based RSA decryption

Worked example. p = 61, q = 53, n = 3233, e = 17, d = 2753 and c = 2790. Precomputed: dp = 2753 mod 60 = 53, dq = 2753 mod 52 = 49, qinv = 53^-1 mod 61 = 38. Then m1 = 2790^53 mod 61 = 4, m2 = 2790^49 mod 53 = 12, h = 38·(4 - 12) mod 61 = 1, and m = 12 + 1·53 = 65, which equals the original message.

Listing 5: CRT decryption with fault check

```
def rsa_dec_crt(c, key):
    p, q = key["p"], key["q"]
    m1 = mod_exp(c % p, key["dp"], p)
    m2 = mod_exp(c % q, key["dq"], q)
    h = key["qinv"] * (m1 - m2) % p
    m = m2 + h * q
    if mod_exp(m, key["e"], key["n"]) != c % key["n"]:   # guards against CRT fault attack
        raise ValueError("CRT fault detected")
    return m
```

Security note. If a hardware or software fault corrupts only one of the two half-computations, the faulty result can reveal a prime factor of n (the Bellcore fault attack). The corrected code therefore re-encrypts the result and compares it with c before releasing it.

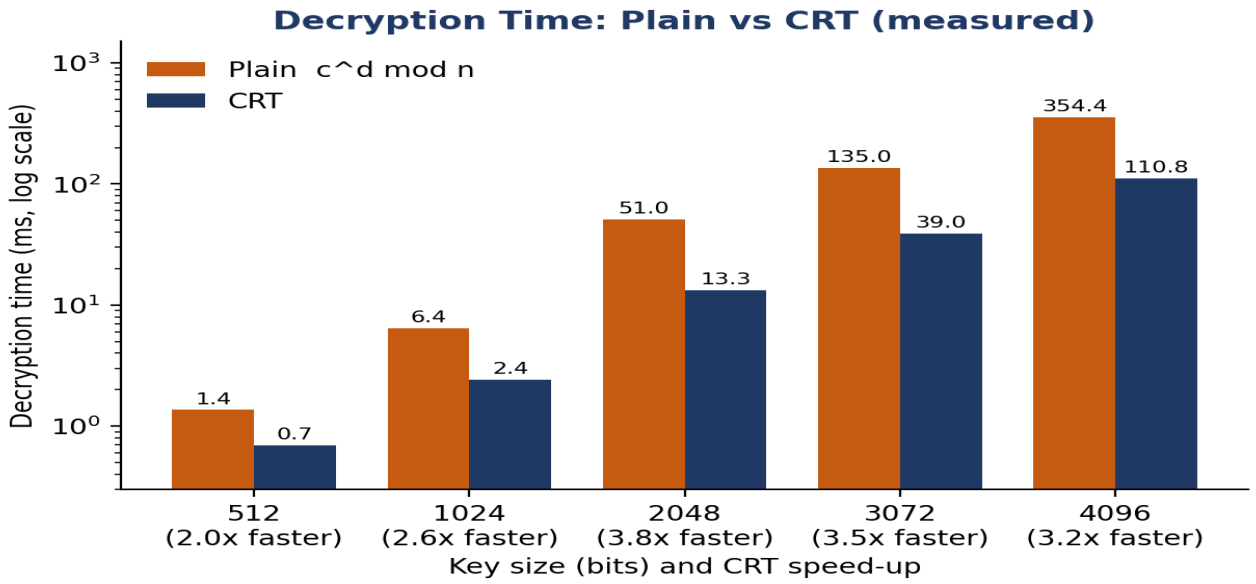


Figure 4: Measured decryption time, plain versus CRT (pure Python)

CRT reduced decryption time by 2.0x at 512 bits and 3.8x at 2048 bits (51.0 ms down to 13.3 ms). The measured gain is slightly below the theoretical four times because of interpreter overhead and the extra recombination step.

10. Padding and Handling Large Messages

RSA can only encrypt an integer smaller than n, so a message must be shortened or split. Raw ('textbook') RSA is also unsafe: it is deterministic (equal plaintexts give equal ciphertexts), malleable, and vulnerable to the small-e attack shown earlier. The corrected code uses RSAES-OAEP with SHA-256, which adds random padding, guarantees the integer is below n, encodes the message length (which fixes the leading-zero problem), and makes the ciphertext randomised.

- **Maximum message per block:** k - 2·hLen - 2 bytes, where k is the modulus length in bytes and hLen = 32. This is 190 bytes for a 2048-bit key and 62 bytes for a 1024-bit key. Longer inputs raise an error instead of silently producing wrong output.
- **Chunking works but is inefficient:** encrypting 100,000 bytes with a 2048-bit key needed 527 RSA blocks, took 8.88 s to decrypt and expanded the data to 134,912 bytes.
- **Hybrid encryption is the recommended design:** RSA-OAEP protects only a random 256-bit AES key, and AES-256-GCM encrypts the data. The same 100,000 bytes took 3.0 ms to encrypt and 18.2 ms to decrypt, with only 284 bytes of overhead.

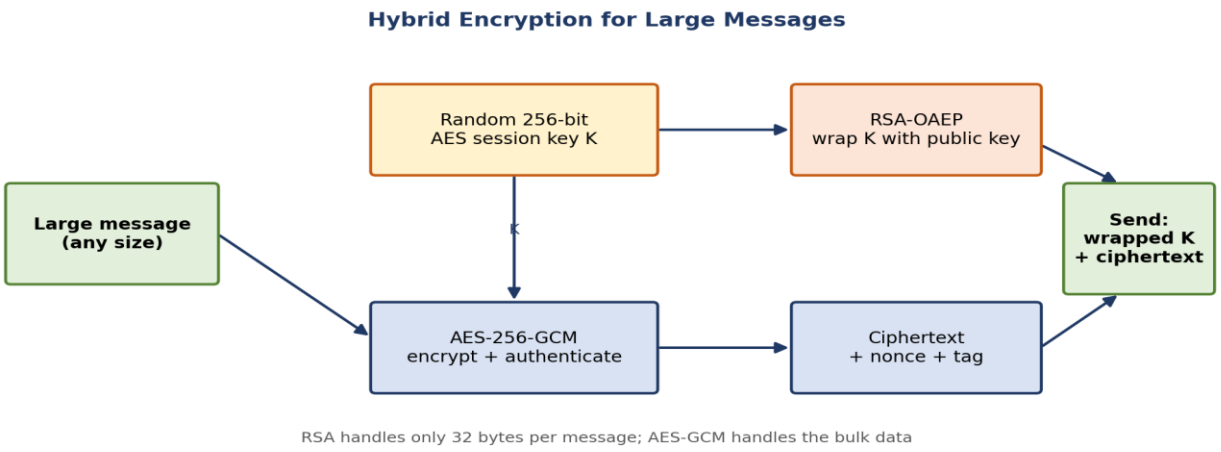


Figure 5: Hybrid RSA-OAEP and AES-GCM encryption for large messages

Listing 6: OAEP block encryption and decryption

```
def encrypt_block(msg, pub):
    e, n = pub; k = (n.bit_length() + 7) // 8
    return mod_exp(int.from_bytes(oaep_encode(msg, k), "big"), e, n).to_bytes(k, "big")

def decrypt_block(ct, key, crt=True):
    n = key["n"]; k = (n.bit_length() + 7) // 8
    c = int.from_bytes(ct, "big")
    if c >= n: raise ValueError("ciphertext out of range")
    m = (rsa_dec_crt if crt else rsa_dec_plain)(c, key)
    return oaep_decode(m.to_bytes(k, "big"), k)
```

11. Verification of the Corrected Implementation

Table 6: Test results for the corrected implementation

Test	Result
Random round trips (768-bit and 1024-bit keys, random lengths up to the block limit)	200 of 200 correct
Message with leading zero bytes (00 00 48 69)	Recovered exactly
Long message (5,000 bytes, 81 OAEP blocks)	Recovered exactly
Same plaintext encrypted twice	Different ciphertexts (randomised padding)

Test	Result
Encrypt with this code, decrypt with the cryptography library (OAEP, SHA-256)	Passed
Encrypt with the cryptography library, decrypt with this code	Passed
Composite and non-invertible-e keys	Rejected during key generation

Table 7: Summary of corrections

Area	Before	After
Primality	Fermat test, base 2	Trial division + Miller-Rabin (16 rounds, CSPRNG)
Prime properties	p may equal or be close to q	Distinct, well separated, exact bit length
Public exponent	e = 3, unchecked	e = 65537, gcd(e, lambda) = 1 verified
Private exponent	Brute-force loop	Extended Euclid, large d enforced
Exponentiation	(m ** e) % n	Square-and-multiply with reduction at every step
Decryption	Single full-size exponentiation	CRT with fault check
Message encoding	Raw integer, no padding	OAEP; hybrid AES-GCM for large data

Part 3: Key Size, Security and Performance Trade-offs

12. Measured Cost of Each Key Size

The corrected code was benchmarked on the same machine for several key sizes. Times are averages of repeated runs in pure Python on this machine, and absolute values will differ on other systems, but the trends hold in general.

Table 8: Measured cost versus security by key size

Key size (bits)	Security (bits, approx.)	Key gen (s)	Encrypt e=65537 (ms)	Decrypt plain (ms)	Decrypt CRT (ms)	Ciphertext size (bytes)
512	~56 (factored in 1999)	0.024	0.02	1.4	0.7	64
1024	~80 (deprecated)	0.126	0.08	6.4	2.4	128
2048	~112	1.3	0.20	51.0	13.3	256
3072	~128	5.1	0.40	135.0	39.0	384
4096	~140	26.0	1.07	354.4	110.8	512

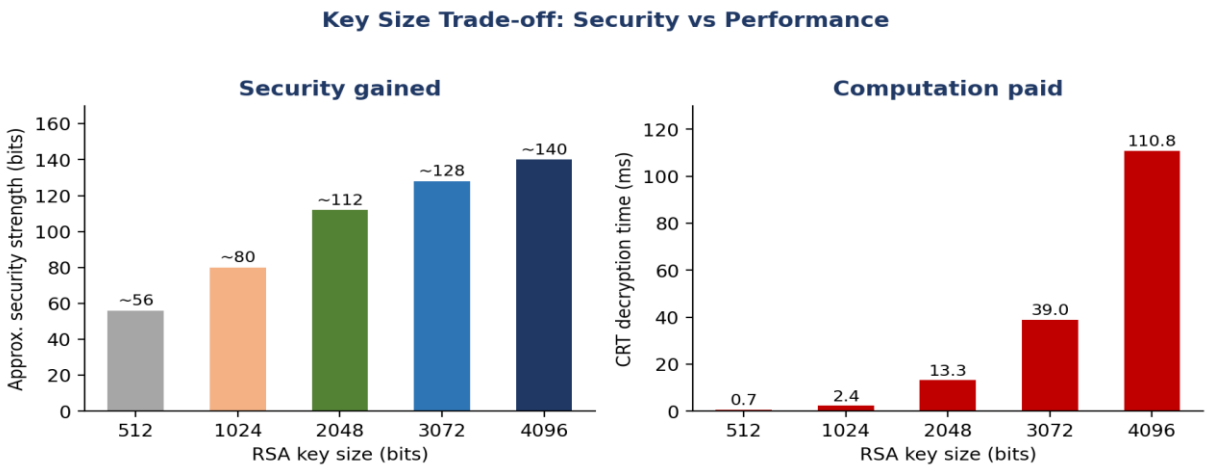


Figure 6: Security strength gained versus decryption time paid

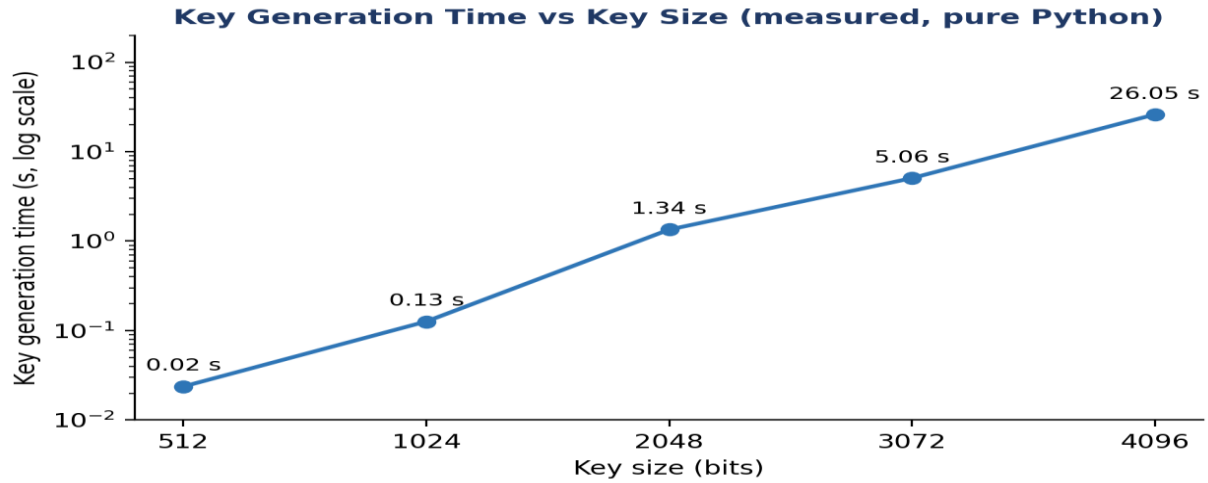


Figure 7: Key generation time by key size (measured)

13. Analysis of the Trade-offs

13.1 Key size versus security

RSA security rests on the difficulty of factoring n. The best known method, the general number field sieve, is sub-exponential, so each additional key bit gives less extra security than in a symmetric cipher. Doubling the modulus from 1024 to 2048 bits raises the strength from about 80 to about 112 bits, and 3072 bits is needed for about 128 bits. Small keys have been broken in practice: 512-bit RSA was factored in 1999 and 768-bit RSA in 2009, so 512-bit keys give no real protection and 1024-bit keys are no longer recommended.

13.2 Key size versus performance

- **Decryption and signing** cost grows roughly with the cube of the key size: doubling from 1024 to 2048 bits increased plain decryption from 6.4 ms to 51.0 ms (about 8.0x).
- **Key generation** is the most expensive operation because two large primes must be found; it grew from 0.13 s at 1024 bits to 1.34 s at 2048 bits and about 26 s at 4096 bits (one run; it varies widely because prime search is random). It is a one-time cost per key pair.
- **Encryption and signature verification** stay cheap because e = 65537 is short: 0.20 ms at 2048 bits, about 67 times faster than CRT decryption.
- **Bandwidth and storage:** ciphertext and signature size equal the key size (256 bytes at 2048 bits, 512 bytes at 4096 bits), which matters for constrained links.

13.3 Role of the optimisations

The optimisations change the trade-off. With CRT, a 3072-bit key decrypts in about the same time as a 2048-bit key without CRT (39.0 ms versus 51.0 ms), so the optimised implementation can afford a stronger key for the same cost. Hybrid encryption removes the key size from the cost of large messages altogether, since RSA is used only once to protect a 32-byte key.

Table 9: Recommended key size by use case

Use case	Suggested RSA size	Reasoning
Legacy or lab testing only	1024 bits	Fast, but about 80-bit security; not recommended for real data
General banking, web and e-mail today	2048 bits	About 112-bit security; considered acceptable by NIST through 2030; good balance of speed and strength
Data that must stay secret for many years	3072 bits	About 128-bit security, matching AES-128; CRT keeps decryption affordable
Very high assurance or long-lived root keys	4096 bits	Extra margin, but decryption is about 8x slower and key generation far slower than 2048 bits
Constrained or high-throughput systems	Consider ECC (256-bit)	About 128-bit security with 32-byte keys and much faster private-key operations

14. Conclusion

The faulty system failed because it accepted pseudoprimes, did not check that e is invertible, used an impractical and imprecise exponentiation method, and encrypted raw integers that could exceed n or lose leading zero bytes. These defects explain both the failures on large messages and the occasionally wrong plaintext. The corrected design uses **Miller-Rabin prime generation, e = 65537 with extended-Euclid inversion, square-and-multiply, CRT decryption with a fault check, and OAEP padding with hybrid encryption for large data**. It passed all 200 random round trips and interoperated with a standard library. On the key-size trade-off, 2048 bits is the practical minimum, 3072 bits is preferable for long-term protection, and the CRT and hybrid techniques keep the extra cost manageable.

References

1. R. Rivest, A. Shamir and L. Adleman, A Method for Obtaining Digital Signatures and Public-Key Cryptosystems, CACM, 1978.
2. RFC 8017, PKCS #1: RSA Cryptography Specifications Version 2.2 (OAEP, CRT private key format).
3. NIST SP 800-57 Part 1 Rev. 5, Recommendation for Key Management (security strengths of key sizes).
4. NIST FIPS 186-5, Digital Signature Standard (prime generation and Miller-Rabin requirements).

5. D. Boneh, Twenty Years of Attacks on the RSA Cryptosystem, Notices of the AMS, 1999 (small exponent, Wiener and fault attacks).
6. W. Stallings, Cryptography and Network Security: Principles and Practice, Pearson.
7. Python documentation: secrets module and the built-in pow(base, exp, mod).

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated

Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation